LEKCJA 06

Lekcja 06: Kolekcje II – Słowniki, zbiory i krotki

#lekcja #datascience #python #kolekcje #slowniki #zbiory #krotki

LearnIT

learnit.com.pl

Spis treści

1. Słowniki (Dictionaries)
2. Zbiory (Sets)
3. Krotki (Tuples)
4. Wybór odpowiedniej kolekcji
5. 🌱 Aspekt środowiskowy
6. 📊 Praca z prawdziwymi danymi
7. 🧪 Zadania praktyczne
8. 📚 Podsumowanie
9. 📖 Dodatkowe zasoby

W poprzedniej lekcji poznaliśmy listy i łańcuchy znaków. Teraz uzupełnimy zestaw kluczowych kolekcji w Pythonie o trzy kolejne typy: **słowniki (dictionaries)**, **zbiory (sets)** oraz **krotki (tuples)**. Każda z tych struktur ma swoje mocne strony w Data Science – słowniki świetnie nadają się do przechowywania par klucz-wartość (np. metadanych modelu), zbiory pozwalają szybko znaleźć elementy unikalne lub wspólne między dwoma grupami, a krotki chronią dane przed przypadkową modyfikacją.

W tej lekcji nauczymy się tworzyć, modyfikować i łączyć te struktury – korzystając wyłącznie z narzędzi, które już znamy z lekcji 1–5 (operatory, liczby, stringi, listy, f-stringi, funkcje wbudowane jak `len`, `min`,

`max`, `sum`, `sorted`).

> 📝 **Notatka**
>
> Pętle (`for`, `while`), instrukcje warunkowe (`if`, `else`), funkcje (`def`) oraz list/dict comprehensions poznamy w lekcjach 7–8. W tej lekcji pokażemy, że słowniki, zbiory i krotki są potężne same w sobie – bez konieczności używania pętli i warunków.

---

1. Słowniki (Dictionaries)

> 📚 **Definicja**
>
> **Słownik** (ang. *dictionary*) to mutowalna kolekcja par **klucz–wartość** (*key–value*). Klucze muszą być unikalne i niemutowalne (najczęściej stringi, liczby lub krotki), a wartości mogą być dowolnego typu – liczby, stringi, listy, a nawet inne słowniki.

Wprowadzenie

Słowniki są jedną z najczęściej używanych struktur danych w Pythonie i Data Science. W przeciwieństwie do list, gdzie dostajemy się do elementów przez **indeks numeryczny** (`lista[0]`), w słownikach odwołujemy się do elementów przez **klucz** (`slownik['imie']`). Dzięki temu słowniki świetnie nadają się do reprezentowania **rekordów** – np. pojedynczego wiersza w tabeli, zestawu metadanych albo konfiguracji modelu ML.

Typowe zastosowania w Data Science:

- metadane datasetu (nazwa, rozmiar, liczba cech),
- wyniki eksperymentu (accuracy, precision, recall, F1),
- hiperparametry modelu (learning rate, liczba epok, batch size),
- reprezentacja rekordu danych – podobnie jak obiekt JSON,
- mapowanie kategorii na wartości numeryczne (np. `{'setosa': 0, 'versicolor': 1, 'virginica': 2}`).

1.1. Tworzenie słowników

Słownik możemy utworzyć na kilka sposobów: literałem `{}`, funkcją `dict()` albo metodą `dict.fromkeys()`.

```
# Pusty słownik
pusty = {}
pusty_tez = dict()

# Słownik z danymi – literał
student = {
    'imie': 'Anna',
    'nazwisko': 'Kowalska',
    'wiek': 25,
    'aktywny': True
}

# Ten sam słownik przez funkcję dict() (klucze jako nazwy argumentów – tylko dla stringów)
student2 = dict(imie='Anna', nazwisko='Kowalska', wiek=25, aktywny=True)

# Słownik z listy par (klucz, wartość) – każdy element to krotka 2-elementowa
pary = [('accuracy', 0.95), ('precision', 0.92), ('recall', 0.89)]
metryki = dict(pary)

# Słownik ze wspólną wartością domyślną dla wielu kluczy
klucze = ['accuracy', 'precision', 'recall', 'f1']
puste_metryki = dict.fromkeys(klucze, 0.0)
# {'accuracy': 0.0, 'precision': 0.0, 'recall': 0.0, 'f1': 0.0}

print(student)
print(metryki)
print(puste_metryki)
```

> 💡 **Wskazówka**
>
> Kluczem może być dowolny obiekt **niemutowalny** – string, liczba, bool, krotka. **Listą ani słownikiem** zwykłym kluczem być nie mogą, bo są mutowalne.

1.2. Dostęp do wartości: [] vs .get()

Są dwa podstawowe sposoby odczytu wartości ze słownika: nawiasami kwadratowymi i metodą `.get()`.

```
student = {'imie': 'Anna', 'wiek': 25, 'email': 'anna@example.com'}

# Dostęp przez nawiasy kwadratowe
print(student['imie'])   # Anna
print(student['wiek'])   # 25

# Próba odczytu nieistniejącego klucza -> KeyError
# print(student['miasto'])  # KeyError: 'miasto'

# Metoda .get() – bezpieczniejsza, zwraca None gdy klucza nie ma
print(student.get('imie'))       # Anna
print(student.get('miasto'))     # None (bez błędu)
print(student.get('miasto', 'Warszawa'))  # Warszawa – wartość domyślna
```

> ⚠ **Uwaga**
>
> Dostęp przez `slownik['klucz']` rzuca wyjątek `KeyError`, jeśli klucza nie ma. `.get()` zwraca `None` (lub wartość domyślną) – używaj go, gdy nie masz pewności, czy klucz istnieje.

1.3. Dodawanie, modyfikacja i usuwanie

Słownik jest **mutowalny** – możemy go zmieniać w miejscu.

```
model = {'nazwa': 'LogisticRegression', 'accuracy': 0.78}

# Dodanie nowego klucza
model['precision'] = 0.81
model['recall'] = 0.75

# Modyfikacja istniejącej wartości (nadpisanie)
model['accuracy'] = 0.82

# Usunięcie klucza – operator del
del model['recall']
```

```
# Usunięcie klucza z pobraniem wartości – metoda .pop()
precision = model.pop('precision')      # 0.81
print(precision)

# Bezpieczne usunięcie z wartością domyślną (gdy klucza nie ma, .pop() też rzuca KeyError
bez drugiego argumentu)
f1 = model.pop('f1_score', None)        # None – bez błędu

# Wyczyszczenie całego słownika
kopia = model.copy()
kopia.clear()
print(kopia)  # {}
print(model)  # {'nazwa': 'LogisticRegression', 'accuracy': 0.82}
```

1.4. Operator in oraz sprawdzanie obecności klucza

Operator `in` sprawdza, czy dany **klucz** (nie wartość!) istnieje w słowniku.

```
parametry = {'lr': 0.01, 'epochs': 100, 'batch_size': 32}

print('lr' in parametry)          # True
print('momentum' in parametry)    # False
print('momentum' not in parametry)  # True

# Uwaga: 'in' sprawdza klucze, nie wartości
print(0.01 in parametry)          # False – 0.01 to wartość, nie klucz
print(0.01 in parametry.values()) # True  – sprawdzamy wartości jawnie
```

1.5. Podstawowe metody: .keys(), .values(), .items()

Te trzy metody zwracają tzw. **widoki** (*views*) – specjalne obiekty reprezentujące odpowiednio klucze, wartości i pary klucz-wartość.

```
wyniki = {'accuracy': 0.95, 'precision': 0.92, 'recall': 0.89}

klucze = list(wyniki.keys())     # ['accuracy', 'precision', 'recall']
wartosci = list(wyniki.values()) # [0.95, 0.92, 0.89]
pary = list(wyniki.items())      # [('accuracy', 0.95), ('precision', 0.92), ('recall',
0.89)]
```

```
print(klucze)
print(wartosci)
print(pary)

# Szybkie statystyki z wartości – działa od razu na views
print(f"Liczba metryk: {len(wyniki)}")             # 3
print(f"Najwyższa wartość: {max(wyniki.values())}")  # 0.95
print(f"Najniższa wartość: {min(wyniki.values())}")  # 0.89
print(f"Suma metryk: {sum(wyniki.values()):.2f}")    # 2.76
print(f"Średnia: {sum(wyniki.values()) / len(wyniki):.3f}")  # 0.920
```

> 💡 **Wskazówka**
>
> Widoki `.keys()`, `.values()`, `.items()` są **dynamiczne** – jeśli zmienisz słownik, widok od razu będzie to odzwierciedlał. W większości przypadków po prostu konwertujemy je na listę (`list(...)`) lub przekazujemy do funkcji typu `len`, `max`, `min`, `sum`, `sorted`.

1.6. Łączenie słowników: .update() i operator |

Dwa słowniki można połączyć w jeden, aktualizując wartości nakładających się kluczy.

```
domyslne = {'lr': 0.01, 'epochs': 100, 'optimizer': 'sgd'}
uzytkownika = {'lr': 0.001, 'batch_size': 64}

# Metoda .update() – modyfikuje słownik w miejscu
konfiguracja = domyslne.copy()
konfiguracja.update(uzytkownika)
print(konfiguracja)
# {'lr': 0.001, 'epochs': 100, 'optimizer': 'sgd', 'batch_size': 64}

# Operator | (Python 3.9+) – tworzy nowy słownik
konfiguracja2 = domyslne | uzytkownika
print(konfiguracja2)
# {'lr': 0.001, 'epochs': 100, 'optimizer': 'sgd', 'batch_size': 64}

# Operator |= (Python 3.9+) – aktualizuje w miejscu, równoważnie z .update()
konfiguracja3 = domyslne.copy()
konfiguracja3 |= uzytkownika
print(konfiguracja3)
```

> 📝 **Notatka**
>
> Przy łączeniu kolejność ma znaczenie: `a | b` oznacza „weź `a`, a potem nadpisz kluczami z `b`". W przykładzie wyżej `lr z uzytkownika` nadpisuje `lr z domyslne`.

1.7. Zagnieżdżone słowniki

Wartością w słowniku może być **inny słownik** albo **lista**. Takie struktury są świetną reprezentacją danych tabelarycznych i JSON-owych.

```
# Pojedynczy rekord Iris jako słownik
iris_rekord = {
    'sepal_length': 5.1,
    'sepal_width': 3.5,
    'petal_length': 1.4,
    'petal_width': 0.2,
    'species': 'setosa'
}

# Metadane całego datasetu — zagnieżdżenie
dataset_info = {
    'nazwa': 'Iris',
    'liczba_rekordow': 150,
    'cechy': ['sepal_length', 'sepal_width', 'petal_length', 'petal_width'],
    'klasy': {
        'setosa': 50,
        'versicolor': 50,
        'virginica': 50
    },
    'zrodlo': {
        'autor': 'R.A. Fisher',
        'rok': 1936
    }
}

# Dostęp do zagnieżdżonych wartości
print(dataset_info['klasy']['setosa'])      # 50
print(dataset_info['zrodlo']['autor'])      # R.A. Fisher
print(dataset_info['cechy'][0])             # sepal_length
print(len(dataset_info['cechy']))           # 4
```

```
# Formatowanie w raporcie
print(f"Dataset: {dataset_info['nazwa']} ({dataset_info['liczba_rekordow']} rekordów)")
print(f"Autor: {dataset_info['zrodlo']['autor']}, rok {dataset_info['zrodlo']['rok']}")
```

1.8. .setdefault() – wartość domyślna przy pierwszym dostępie

Metoda `.setdefault(klucz, wartosc)` zwraca wartość klucza, a jeśli klucz nie istnieje – dodaje go z zadaną wartością domyślną.

```
konfiguracja = {'lr': 0.01, 'epochs': 100}

# Klucz istnieje -> zwraca istniejącą wartość, słownik bez zmian
print(konfiguracja.setdefault('lr', 0.1))   # 0.01
print(konfiguracja)  # {'lr': 0.01, 'epochs': 100}

# Klucz nie istnieje -> dodaje go i zwraca ustawioną wartość
print(konfiguracja.setdefault('momentum', 0.9))  # 0.9
print(konfiguracja)
# {'lr': 0.01, 'epochs': 100, 'momentum': 0.9}
```

Mini-podsumowanie: metody słowników

Metoda	Działanie
`d[k]`	Odczyt wartości – `KeyError` gdy brak klucza
`d.get(k, default)`	Bezpieczny odczyt – zwraca `default` gdy brak klucza
`d[k] = v`	Dodanie/nadpisanie wartości
`del d[k]`	Usunięcie klucza
`d.pop(k, default)`	Usunięcie z pobraniem wartości

Metoda	Działanie	
`d.keys()` , `d.values()` , `d.items()`	Widoki kluczy, wartości, par	
`d.update(inny)`	Scalenie ze słownikiem `inny` (w miejscu)	
`d1 | d2`	Nowy słownik – scalenie bez mutacji	
`d.setdefault(k, v)`	Odczyt z ustawieniem wartości domyślnej	
`d.copy()`	Płytka kopia	
`d.clear()`	Usuwa wszystkie pary	
`k in d` , `k not in d`	Sprawdzenie obecności **klucza**	

---

2. Zbiory (Sets)

> 📚 **Definicja**
>
> **Zbiór** (ang. *set*) to mutowalna kolekcja **unikalnych** elementów, w której kolejność nie ma znaczenia. Zbiory wspierają klasyczne operacje matematyczne: sumę, iloczyn, różnicę i różnicę symetryczną.

Wprowadzenie

Zbiory są nieocenione, gdy interesuje nas **jakie elementy** występują w kolekcji, ale **nie ich kolejność ani liczność**. Do wyłapywania duplikatów, porównywania grup danych albo szybkiego sprawdzenia, czy coś już się pojawiło – zbiory są najlepszym wyborem.

Zastosowania w Data Science:

- deduplikacja listy wartości (np. unikalne kategorie),
- szybkie sprawdzenie przynależności (*membership test*) – operator `in` w zbiorze jest bardzo szybki,
- znajdowanie wspólnych kolumn między dwoma datasetami,
- porównywanie zbiorów użytkowników, tagów, słów.

2.1. Tworzenie zbiorów

```
# Pusty zbiór – tylko przez set()! {} to pusty SŁOWNIK, nie zbiór
pusty_zbior = set()
pusty_slownik = {}

# Zbiór z listy – duplikaty są automatycznie usuwane
liczby = [1, 2, 3, 3, 2, 1, 4, 5, 5]
unikalne = set(liczby)
print(unikalne)  # {1, 2, 3, 4, 5}

# Zbiór literałem {} – przy niepustej kolekcji
kolory = {'czerwony', 'zielony', 'niebieski'}
print(kolory)

# Zbiór ze stringa – każdy znak staje się elementem
litery = set('abracadabra')
print(litery)  # {'a', 'b', 'r', 'c', 'd'}

# Liczba unikalnych elementów
print(f"Liczba unikalnych znaków: {len(litery)}")  # 5
```

> ⚠ **Uwaga**
>
> Pusty zbiór tworzymy **tylko** funkcją `set()`. Literał `{}` to **pusty słownik**, nie zbiór – łatwo o pomyłkę.

2.2. Dodawanie i usuwanie elementów

```
jezyki = {'python', 'r', 'sql'}

# Dodanie jednego elementu
jezyki.add('julia')
```

```
jezyki.add('python')   # duplikat – zbiór się nie zmienia

# Dodanie wielu elementów (z listy, krotki, innego zbioru, stringa...)
jezyki.update(['scala', 'go'])
jezyki.update(('rust',), {'haskell'})

# Usunięcie – .remove() rzuca KeyError, gdy elementu nie ma
jezyki.remove('go')

# Usunięcie – .discard() nic nie robi, gdy elementu nie ma (bezpieczne)
jezyki.discard('cobol')   # bez błędu
jezyki.discard('scala')

# Usunięcie dowolnego elementu i zwrócenie go (.pop())
dowolny = jezyki.pop()
print(f"Wyciągnięty: {dowolny}")

# Wyczyszczenie
kopia = jezyki.copy()
kopia.clear()
print(kopia)   # set()
```

2.3. Operacje matematyczne na zbiorach

Zbiory wspierają cztery klasyczne operacje mnogościowe. Każda ma wersję **operatorową** i **metodową** – metody dodatkowo akceptują dowolny *iterable* (listę, krotkę), nie tylko zbiór.

```
ds_python = {'numpy', 'pandas', 'matplotlib', 'scikit-learn'}
ds_r = {'dplyr', 'ggplot2', 'tidyr', 'scikit-learn'}   # 'scikit-learn' tylko dla przykładu

# SUMA – wszystkie elementy z obu zbiorów (bez duplikatów)
suma1 = ds_python | ds_r
suma2 = ds_python.union(ds_r)

# ILOCZYN (część wspólna) – elementy występujące w obu
wspolne1 = ds_python & ds_r
wspolne2 = ds_python.intersection(ds_r)

# RÓŻNICA – elementy z lewego zbioru, których NIE ma w prawym
tylko_python1 = ds_python - ds_r
tylko_python2 = ds_python.difference(ds_r)

# RÓŻNICA SYMETRYCZNA – elementy występujące tylko w jednym ze zbiorów
```

```
xor1 = ds_python ^ ds_r
xor2 = ds_python.symmetric_difference(ds_r)

print(f"Suma: {suma1}")
print(f"Wspólne: {wspolne1}")
print(f"Tylko w Pythonie: {tylko_python1}")
print(f"Różnica symetryczna: {xor1}")
```

Graficznie (diagramy Venn'a w formie tabeli):

Operacja	Symbol	Metoda	Opis	
Suma	`A \`	B	`A.union(B)`	Elementy w A lub w B (lub obu)
Iloczyn	`A & B`	`A.intersection(B)`	Elementy jednocześnie w A i w B	
Różnica	`A - B`	`A.difference(B)`	Elementy w A, których nie ma w B	
Różnica symetryczna	`A ^ B`	`A.symmetric_difference(B)`	Elementy w dokładnie jednym z A, B	

2.4. Relacje między zbiorami

```
wszystkie_cechy = {'sepal_length', 'sepal_width', 'petal_length', 'petal_width'}
wybrane = {'petal_length', 'petal_width'}
liscie = {'sepal_length', 'sepal_width'}

# Czy 'wybrane' jest podzbiorem 'wszystkie_cechy'?
print(wybrane.issubset(wszystkie_cechy))    # True
print(wybrane <= wszystkie_cechy)            # True – zapis operatorowy

# Czy 'wszystkie_cechy' jest nadzbiorem 'wybrane'?
```

```
print(wszystkie_cechy.issuperset(wybrane))   # True
print(wszystkie_cechy >= wybrane)            # True

# Czy zbiory są rozłączne (nie mają wspólnych elementów)?
print(wybrane.isdisjoint(liscie))            # True
print(wybrane.isdisjoint(wszystkie_cechy))   # False – mają wspólne

# Równość zbiorów
print({1, 2, 3} == {3, 2, 1})                # True – kolejność nieistotna
```

2.5. Sprawdzanie przynależności

Operator `in` w zbiorze jest bardzo szybki (stała złożoność obliczeniowa, w przeciwieństwie do listy).

```
uzytkownicy_premium = {'user_12', 'user_45', 'user_98', 'user_103', 'user_221'}

print('user_45' in uzytkownicy_premium)    # True
print('user_999' in uzytkownicy_premium)   # False
print('user_999' not in uzytkownicy_premium)  # True
```

> 💡 **Wskazówka**
>
> Jeśli wielokrotnie sprawdzasz, czy coś znajduje się w kolekcji, zamień listę na zbiór. Dla dużych kolekcji operacja `in` w zbiorze jest **dramatycznie** szybsza niż w liście.

Mini-podsumowanie: metody zbiorów

Metoda / operator	Działanie
`s.add(x)`	Dodaje `x`
`s.update(iter)`	Dodaje elementy z `iter`
`s.remove(x)`	Usuwa `x` – `KeyError` gdy nie ma
`s.discard(x)`	Usuwa `x` – brak błędu gdy nie ma

Metoda / operator	Działanie
`s.pop()`	Usuwa i zwraca dowolny element
`s.clear()`	Czyści zbiór
`a \`	b, `a.union(b)` Suma
`a & b` , `a.intersection(b)`	Iloczyn
`a - b` , `a.difference(b)`	Różnica
`a ^ b` , `a.symmetric_difference(b)`	Różnica symetryczna
`a <= b` , `a.issubset(b)`	Podzbiór
`a >= b` , `a.issuperset(b)`	Nadzbiór
`a.isdisjoint(b)`	Rozłączność
`x in s` , `x not in s`	Przynależność

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

3. Krotki (Tuples)

> 📚 **Definicja**
>
> **Krotka** (ang. *tuple*) to **niemutowalna** (po utworzeniu nie można jej zmieniać) sekwencja elementów. Używana do reprezentacji stałych grup powiązanych wartości – np. współrzędnych punktu, rekordu, daty.

Wprowadzenie

Krotki są podobne do list: można je indeksować, slajsować, iterować, a do elementów dostajemy się tak samo (`t[0]` , `t[1:3]`). Różnica jest kluczowa – krotki są **niemutowalne**. Raz utworzonej krotki nie zmodyfikujemy: nie dodamy do niej elementu, nie usuniemy, nie nadpiszemy.

Po co nam niemutowalność?

- **Bezpieczeństwo** – dane, które mają być stałe (np. współrzędne, wymiary obrazu), są chronione przed przypadkową modyfikacją.
- **Klucz słownika lub element zbioru** – mutowalne listy nie mogą być kluczami, ale krotki już tak.
- **Wydajność** – krotki zajmują mniej pamięci niż listy.
- **Sygnał intencji** – krotka mówi czytelnikowi kodu „to jest stały rekord, nie chcę go zmieniać".

Zastosowania w Data Science:

- reprezentacja wymiarów datasetu (np. `shape = (150, 4)`),
- zwracanie wielu wartości z funkcji (np. `min_val, max_val, mean = ...`),
- klucze złożone w słownikach (np. `{(2024, 'Q1'): 123.5, (2024, 'Q2'): 145.2}`),
- współrzędne, punkty, kolory RGB.

3.1. Tworzenie krotek

```
# Krotka kilkuelementowa
punkt = (3.14, 2.71)
rgb = (255, 128, 0)
shape = (150, 4)

# Bez nawiasów – nawiasy są opcjonalne (ale czytelność!)
punkt2 = 3.14, 2.71
```

```
print(type(punkt2))   # <class 'tuple'>

# Krotka jednoelementowa – MUSI mieć przecinek!
nie_krotka = (42)       # to INT – nawiasy tu nic nie znaczą
krotka_jedna = (42,)    # TO jest krotka
print(type(nie_krotka))    # <class 'int'>
print(type(krotka_jedna))  # <class 'tuple'>

# Pusta krotka
pusta = ()
pusta_tez = tuple()

# Krotka z iterable
z_listy = tuple([1, 2, 3])
ze_stringa = tuple('abc')    # ('a', 'b', 'c')

print(shape)
print(rgb)
print(z_listy)
```

> ⚠️ **Uwaga**
>
> Krotka jednoelementowa to `(42,)`, nie `(42)`. Bez przecinka mamy zwykłą liczbę w nawiasach. Klasyczny błąd początkujących!

3.2. Dostęp do elementów (jak w listach)

```
rekord = ('Anna', 'Kowalska', 25, 'Data Scientist')

# Indeksowanie – jak w liście
print(rekord[0])   # Anna
print(rekord[-1])  # Data Scientist

# Slajsowanie – też jak w liście
print(rekord[:2])  # ('Anna', 'Kowalska')
print(rekord[2:])  # (25, 'Data Scientist')

# Długość, min, max, sum (na liczbach)
liczby = (10, 20, 30, 40, 50)
print(len(liczby))   # 5
print(min(liczby))   # 10
```

```
print(max(liczby))   # 50
print(sum(liczby))   # 150

# Operator in
print('Anna' in rekord)     # True
print('Kowalski' in rekord) # False
```

3.3. Niemutowalność

```
punkt = (3.14, 2.71)

# punkt[0] = 0  # TypeError: 'tuple' object does not support item assignment
# punkt.append(1)  # AttributeError: 'tuple' object has no attribute 'append'

# Jedyne „wyjście" – utworzenie nowej krotki
nowy_punkt = (0, punkt[1])
print(nowy_punkt)  # (0, 2.71)

# Konkatenacja tworzy NOWĄ krotkę
rozszerzona = punkt + (100, 200)
print(rozszerzona)  # (3.14, 2.71, 100, 200)

# Powielanie (jak w liście/stringu)
trojka = (0,) * 3
print(trojka)  # (0, 0, 0)
```

3.4. Rozpakowywanie (unpacking) krotek

Rozpakowywanie to bardzo wygodna cecha krotek – pozwala „wyciągnąć" wartości do kilku zmiennych w jednym wierszu.

```
# Rozpakowanie do zmiennych
punkt = (3.14, 2.71)
x, y = punkt
print(f"x = {x}, y = {y}")   # x = 3.14, y = 2.71

# Zamiana miejscami – bez zmiennej tymczasowej!
a, b = 10, 20
a, b = b, a
print(f"a = {a}, b = {b}")   # a = 20, b = 10
```

```python
# Rozpakowanie rekordu
rekord = ('Anna', 'Kowalska', 25, 'Data Scientist')
imie, nazwisko, wiek, stanowisko = rekord
print(f"{imie} {nazwisko}, {wiek} lat, {stanowisko}")

# Rozpakowanie z gwiazdką – reszta trafia do listy
liczby = (1, 2, 3, 4, 5)
pierwszy, *srodek, ostatni = liczby
print(pierwszy)  # 1
print(srodek)    # [2, 3, 4]
print(ostatni)   # 5

# Ignorowanie niechcianych wartości (konwencja: podkreślnik)
_, _, wiek, _ = rekord
print(f"Wiek: {wiek}")
```

3.5. Metody krotek

Krotki mają tylko **dwie** metody – bo to kolekcja niemutowalna, nie ma metod modyfikujących.

```python
oceny = (4.5, 5.0, 3.0, 4.5, 5.0, 4.5, 2.0, 5.0)

# .count() – ile razy element występuje
print(oceny.count(5.0))   # 3
print(oceny.count(4.5))   # 3
print(oceny.count(1.0))   # 0

# .index() – pozycja pierwszego wystąpienia
print(oceny.index(5.0))   # 1
print(oceny.index(2.0))   # 6
# print(oceny.index(10))  # ValueError – nie ma elementu
```

3.6. Krotki jako klucze słownika

Ponieważ krotki są niemutowalne i hashowalne, mogą służyć jako **klucze** w słownikach – listy nie mogą.

```python
# Sprzedaż w podziale na rok i kwartał – klucz to (rok, kwartał)
sprzedaz = {
    (2024, 'Q1'): 123_000,
    (2024, 'Q2'): 145_000,
```

```
    (2024, 'Q3'): 138_000,
    (2024, 'Q4'): 167_000,
    (2025, 'Q1'): 131_000
}

# Dostęp po kluczu-krotce
print(sprzedaz[(2024, 'Q2')])          # 145000
print(sprzedaz.get((2024, 'Q1')))      # 123000

# Szybkie statystyki
print(f"Kwartałów w danych: {len(sprzedaz)}")
print(f"Najwyższa sprzedaż: {max(sprzedaz.values())}")
print(f"Suma: {sum(sprzedaz.values())}")
```

> 💡 **Wskazówka**
>
> Krotka jako klucz jest wzorcem często spotykanym w analizie danych panelowych (wymiar czasowy + wymiar kategoryczny) – zanim poznamy DataFrame'y z Pandas, jest to pełnoprawna reprezentacja „tabeli z dwuwymiarowym indeksem".

Mini-podsumowanie: krotki

Operacja	Przykład	Uwagi
Tworzenie	`t = (1, 2, 3)` lub `t = 1, 2, 3`	Przecinek decyduje
Jednoelementowa	`t = (1,)`	**Wymagany** przecinek
Pusta	`t = ()` albo `tuple()`	
Dostęp	`t[0]` , `t[-1]` , `t[1:3]`	Jak w liście
Długość	`len(t)`	
`in` / `not in`	`5 in t`	Sprawdza wartość
Konkatenacja	`t1 + t2`	Zwraca nową krotkę

Operacja	Przykład	Uwagi
Powielenie	`t * 3`	Zwraca nową krotkę
Rozpakowanie	`a, b, c = t`	Liczba zmiennych = długość
`.count(x)`	`t.count(5)`	Liczba wystąpień
`.index(x)`	`t.index(5)`	Pozycja pierwszego
Niemutowalność	`t[0] = 10` ❌	`TypeError`

---

4. Wybór odpowiedniej kolekcji

Masz już w rękach cztery kolekcje: **lista, słownik, zbiór, krotka**. Ich wybór w praktyce jest podyktowany charakterem danych.

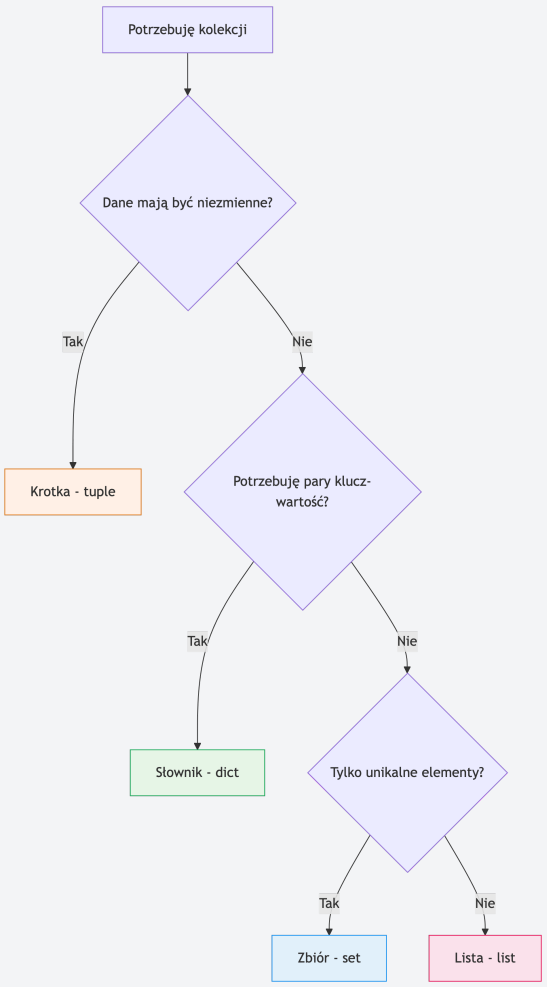


Diagram 1

Porównanie w jednej tabeli

Cecha	Lista	Krotka	Słownik	Zbiór
Składnia literału	`[1, 2, 3]`	`(1, 2, 3)`	`{'a': 1}`	`{1, 2, 3}`
Uporządkowana?	Tak	Tak	Tak (od Pythona 3.7)	Nie
Duplikaty?	Tak	Tak	Klucze – nie, wartości – tak	Nie

Cecha	Lista	Krotka	Słownik	Zbiór
Mutowalna?	Tak	Nie	Tak	Tak
Dostęp przez	indeks	indeks	klucz	brak indeksu (`in`)
Typowe zastosowanie	sekwencja, lista rekordów	rekord, współrzędne	mapowanie, JSON-like	unikalne elementy, `in`

Kiedy co wybrać – praktycznie

Chcesz...	Wybierz
...przechowywać posortowane pomiary temperatury	**lista**
...reprezentować współrzędne punktu `(x, y)`	**krotka**
...mapować nazwę miasta → populację	**słownik**
...znaleźć unikalne kategorie w kolumnie	**zbiór**
...iterować rekordy (w lekcji 7)	**lista krotek** lub **lista słowników**
...szybko sprawdzać „czy user już klikał?"	**zbiór**
...zwracać wiele wartości z funkcji (lekcja 8)	**krotka**

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

5. 🌱 Aspekt środowiskowy

Wybór struktury danych to również decyzja o zużyciu pamięci i mocy obliczeniowej – a więc o **zużyciu energii**. W skali pojedynczego skryptu to niewielkie liczby, ale w Data Science pracujemy z milionami rekordów i modelami trenowanymi godzinami.

Pamięć

Przy tym samym zestawie elementów:

- **krotka** zajmuje mniej pamięci niż **lista** (ok. 50–60 bajtów mniej przy 5 elementach) – bo jest niemutowalna i nie rezerwuje zapasu pod dopisywanie,
- **zbiór** i **słownik** używają tablicy hash, więc na jeden element zużywają więcej pamięci niż lista/krotka, ale za to sprawdzenie `in` jest niemal natychmiastowe,
- **lista** rezerwuje pewien „zapas" (capacity), co przyspiesza `.append()`, ale zużywa nieco więcej pamięci niż wynikałoby to z liczby elementów.

Wydajność

Operacja	Lista	Krotka	Słownik (po kluczu)	Zbiór
Dostęp do elementu	szybki (O(1))	szybki (O(1))	bardzo szybki (O(1))	–
`x in kolekcja`	wolny (O(n))	wolny (O(n))	bardzo szybki (O(1))	bardzo szybki (O(1))
Dodanie elementu	szybkie (amortyzowane)	niemożliwe	szybkie	szybkie

> 💡 **Wskazówka**
>
> **Praktyczna reguła dla zielonego kodu:** jeśli piszesz skrypt, który wielokrotnie pyta „czy ten element już wystąpił?", trzymaj dane w zbiorze, nie w liście. W skali miliona zapytań różnica to minuty vs. sekundy obliczeń – i proporcjonalnie mniejsze zużycie energii serwera.

Dobre praktyki „green computing"

- Używaj **krotek** zamiast list, jeśli dane są stałe – oszczędność pamięci bez wysiłku.
- Deduplikuj listy przez `set(lista)` zamiast wymyślać własne algorytmy – biblioteka wbudowana jest zoptymalizowana.
- Przy łączeniu zbiorów/słowników korzystaj z operatorów `|`, `&`, `-`, `^` – są krótsze, czytelniejsze i równie szybkie co metody.
- Używaj `.get(klucz, domyslne)` zamiast dwóch zapytań `if klucz in d: ... else: ...` (`if` poznamy w lekcji 7, ale już teraz `.get()` jest elegantszy).

---

6. 📊 Praca z prawdziwymi danymi

Połączmy trzy poznane struktury w realistycznym scenariuszu analitycznym – bez ani jednej pętli i ani jednego `if`.

Scenariusz: eksperyment ML na datasetie Iris

Chcemy mieć w jednym miejscu:

- metadane datasetu (słownik z zagnieżdżonymi strukturami),
- wyniki kilku modeli (słownik, którego wartościami są słowniki metryk),
- zbiór unikalnych cech używanych w eksperymencie,
- krotkę reprezentującą wymiary (150 rekordów × 4 cechy).

```
# --- Metadane datasetu ---
iris = {
    'nazwa': 'Iris',
    'shape': (150, 4),                          # krotka! (n_rekordów, n_cech)
    'cechy': ('sepal_length', 'sepal_width',
              'petal_length', 'petal_width'),    # krotka – stała struktura
    'klasy': ('setosa', 'versicolor', 'virginica')
}

# --- Wyniki trzech modeli ---
wyniki = {
    'LogisticRegression': {
        'accuracy': 0.9667,
        'precision': 0.9673,
        'recall': 0.9667,
        'f1': 0.9667
    },
    'DecisionTree': {
        'accuracy': 0.9333,
        'precision': 0.9342,
        'recall': 0.9333,
        'f1': 0.9332
    },
    'RandomForest': {
        'accuracy': 0.9600,
        'precision': 0.9610,
        'recall': 0.9600,
        'f1': 0.9601
    }
}

# --- Szybki raport ---
n_rekordow, n_cech = iris['shape']   # rozpakowanie krotki!
print(f"Dataset {iris['nazwa']}: {n_rekordow} rekordów × {n_cech} cechy")
```

```
print(f"Klasy: {', '.join(iris['klasy'])}")
print(f"Liczba modeli: {len(wyniki)}")

# --- Dostęp do pojedynczych metryk ---
lr = wyniki['LogisticRegression']
print(f"\nLogisticRegression -> accuracy = {lr['accuracy']:.4f}, f1 = {lr['f1']:.4f}")

# --- Porównanie accuracy wszystkich modeli ---
acc_values = [
    wyniki['LogisticRegression']['accuracy'],
    wyniki['DecisionTree']['accuracy'],
    wyniki['RandomForest']['accuracy']
]
print(f"\nAccuracy: min={min(acc_values):.4f}, "
      f"max={max(acc_values):.4f}, "
      f"średnia={sum(acc_values) / len(acc_values):.4f}")

# --- Zbiór unikalnych cech w eksperymencie ---
cechy_uzyte = set(iris['cechy'])
cechy_wybrane = {'petal_length', 'petal_width'}          # tylko płatki
cechy_pominiete = cechy_uzyte - cechy_wybrane            # różnica zbiorów

print(f"\nWszystkich cech: {cechy_uzyte}")
print(f"Wybrane (płatki): {cechy_wybrane}")
print(f"Pominięte (sepal): {cechy_pominiete}")
```

Scenariusz: deduplikacja odpowiedzi ankietowych

```
# Surowe odpowiedzi – wielu respondentów mogło wybrać te same języki
jezyki_respondent_1 = ['Python', 'SQL', 'R']
jezyki_respondent_2 = ['Python', 'Julia', 'SQL']
jezyki_respondent_3 = ['R', 'SQL', 'Python']
jezyki_respondent_4 = ['Python', 'SQL']

# Wszystkie wybory w jednym zbiorze – unikalne języki
wszystkie = set(jezyki_respondent_1) | set(jezyki_respondent_2) \
            | set(jezyki_respondent_3) | set(jezyki_respondent_4)

print(f"Unikalne języki w ankiecie: {wszystkie}")
print(f"Liczba różnych języków: {len(wszystkie)}")

# Część wspólna wyborów respondenta 1 i 2
wspolne_1_2 = set(jezyki_respondent_1) & set(jezyki_respondent_2)
```

```
print(f"Respondent 1 i 2 wspólnie wskazali: {wspolne_1_2}")

# Respondent 1 - języki, których nie wybrał respondent 4
tylko_1 = set(jezyki_respondent_1) - set(jezyki_respondent_4)
print(f"Tylko respondent 1 (a nie 4): {tylko_1}")

# Czy ktoś wybrał Julię? (szukamy w sumie)
print(f"Ktoś wybrał Julię? {'Julia' in wszystkie}")          # True
print(f"Ktoś wybrał Fortran? {'Fortran' in wszystkie}")      # False
```

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

7. 🧪 Zadania praktyczne

> 📝 Notatka
>
> Wszystkie zadania rozwiążesz używając **wyłącznie** narzędzi z lekcji 1–6: operatorów, list, stringów, f-stringów, słowników, zbiorów, krotek oraz funkcji wbudowanych `len`, `min`, `max`, `sum`, `sorted`. **Bez** pętli, `if`, `lambda` czy funkcji własnych.

🟢 Zadania podstawowe

Zadanie 1. Pierwszy słownik studenta

Utwórz słownik `student` zawierający klucze: `imie` (`'Kamil'`), `nazwisko` (`'Nowak'`), `wiek` (`22`), `kierunek` (`'Informatyka'`). Następnie:

1. Dodaj klucz `email` o wartości `'kamil.nowak@edu.pl'`.
2. Zmień wartość klucza `wiek` na `23`.
3. Używając f-stringa wypisz: `Student: Kamil Nowak (23 lata), kierunek: Informatyka, email: kamil.nowak@edu.pl`.
4. Wypisz listę kluczy i listę wartości (użyj `list()` na `.keys()` i `.values()`).

Zadanie 2. Bezpieczny dostęp do słownika

Dany jest słownik:

```
konfig = {'lr': 0.01, 'epochs': 100, 'optimizer': 'adam'}
```

1. Pobierz wartość klucza `'lr'` przez `.get()` – wypisz ją.
2. Pobierz wartość klucza `'momentum'` przez `.get()` – wypisz wynik (powinno być `None`).
3. Pobierz wartość klucza `'momentum'` przez `.get()` z wartością domyślną `0.9` – wypisz ją.
4. Sprawdź, czy `'batch_size'` jest kluczem słownika (operator `in`) – wypisz wynik.

Zadanie 3. Metryki modelu

Utwórz słownik `metryki` z kluczami `accuracy`, `precision`, `recall`, `f1_score` i wartościami odpowiednio `0.94`, `0.91`, `0.89`, `0.90`. Następnie używając funkcji wbudowanych (`min`, `max`, `sum`, `len`, `sorted`):

1. Wypisz najwyższą i najniższą wartość metryki.
2. Wypisz średnią metrykę z dokładnością do 3 miejsc po przecinku.
3. Wypisz klucze posortowane alfabetycznie (`sorted`).
4. Wypisz wartości posortowane rosnąco.

Zadanie 4. Zbiór unikalnych języków

Dana jest lista:

```
jezyki = ['Python', 'R', 'SQL', 'Python', 'Julia', 'R', 'Python', 'Scala', 'SQL']
```

1. Utwórz zbiór `unikalne` z tej listy.
2. Wypisz liczbę unikalnych języków.
3. Dodaj do zbioru `'Go'` i `'Rust'` (użyj `.update()`).
4. Sprawdź (operatorem `in`), czy w zbiorze są: `'Python'`, `'Fortran'`, `'Julia'`.
5. Usuń `'Scala'` metodą `.discard()` (bezpieczną, bez błędu gdyby go nie było).
6. Wypisz finalny zbiór oraz jego długość.

Zadanie 5. Operacje na dwóch zbiorach

Dane są dwa zbiory narzędzi używanych przez dwie grupy analityków:

```
grupa_a = {'python', 'pandas', 'numpy', 'sql', 'tableau'}
grupa_b = {'python', 'r', 'sql', 'ggplot2', 'dplyr'}
```

Używając **zarówno** operatorów (`|`, `&`, `-`, `^`), jak i odpowiadających im metod, wypisz:

1. Sumę zbiorów (wszystkie narzędzia).
2. Wspólne narzędzia (iloczyn).
3. Narzędzia tylko w grupie A (różnica `grupa_a - grupa_b`).
4. Narzędzia tylko w jednej z grup (różnica symetryczna).
5. Czy zbiory są rozłączne? (`.isdisjoint()`)

Zadanie 6. Pierwsza krotka

1. Utwórz krotkę `wymiary = (1920, 1080)`.
2. Rozpakuj ją do dwóch zmiennych `szerokosc` i `wysokosc` i wypisz: `Rozdzielczość: 1920x1080`.
3. Utwórz krotkę jednoelementową `samotny = (42,)` i wypisz jej typ (`type(samotny)`).
4. Spróbuj wykonać `samotny[0] = 100` – zakomentuj tę linijkę i w komentarzu napisz, jaki błąd byłby rzucony.
5. Utwórz nową krotkę `samotny + (100, 200)` i wypisz ją.

Zadanie 7. Rozpakowanie rekordu

Dana jest krotka opisująca pracownika:

```
pracownik = ('Julia', 'Wiśniewska', 28, 'Data Analyst', 8500.0)
```

1. Rozpakuj wszystkie pola do zmiennych `imie, nazwisko, wiek, stanowisko, pensja`.
2. Wypisz w f-stringu: `Julia Wiśniewska (28) – Data Analyst, pensja: 8500.00 zł`.
3. Używając rozpakowania z gwiazdką, wydziel pierwsze pole jako `imie` i wszystkie pozostałe jako listę `reszta`.
4. Wypisz `imie` i `reszta`.

Zadanie 8. Krotka jako klucz słownika

1. Utwórz słownik `sprzedaz`, którego kluczami są krotki `(rok, kwartał)`, a wartościami liczby:

```
sprzedaz = {
    (2024, 'Q1'): 120_000,
    (2024, 'Q2'): 140_000,
    (2024, 'Q3'): 135_000,
    (2024, 'Q4'): 160_000
}
```

1. Wypisz sprzedaż z `(2024, 'Q2')`.
2. Dodaj wpis `(2025, 'Q1'): 130_000`.
3. Wypisz najwyższą, najniższą i łączną sprzedaż (funkcje wbudowane na `.values()`).
4. Wypisz liczbę różnych kwartałów w słowniku (`len`).

🟡 Zadania średnio zaawansowane

Zadanie 9. Konfiguracja modelu: łączenie słowników

Dane:

```
domyslne = {'lr': 0.01, 'epochs': 100, 'optimizer': 'sgd', 'batch_size': 32}
uzytkownika = {'lr': 0.001, 'batch_size': 64, 'momentum': 0.9}
```

1. Stwórz finalną konfigurację łącząc słowniki operatorem `|` (wartości z `uzytkownika` mają wygrać).
2. Stwórz drugą wersję finalnej konfiguracji używając `.update()` (pamiętaj o `.copy()`, żeby nie zmienić `domyslne`).

3. Sprawdź, że obie wersje są równe (operator `==`).
4. Wypisz finalną konfigurację w postaci: `klucz: wartość` dla wybranych trzech kluczy (`lr`, `batch_size`, `momentum`) – każdy w nowej linii (użyj `\n` w f-stringu).

Zadanie 10. Porównanie cech dwóch datasetów

Dwa datasety mają następujące cechy (kolumny):

```
dataset_a = {'wiek', 'plec', 'miasto', 'dochod', 'wyksztalcenie', 'stan_cywilny'}
dataset_b = {'wiek', 'plec', 'miasto', 'pensja', 'stanowisko', 'staz'}
```

Używając operacji na zbiorach wylicz i wypisz:

1. Cechy wspólne dla obu datasetów (iloczyn).
2. Cechy tylko w `dataset_a` (różnica `a - b`).
3. Cechy tylko w `dataset_b` (różnica `b - a`).
4. Wszystkie unikalne cechy (suma).
5. Cechy, które są tylko w jednym datasecie (różnica symetryczna).
6. Sprawdź, czy `{'wiek', 'plec'}` jest podzbiorem cech wspólnych (`.issubset()`).

Zadanie 11. Mini-raport z zagnieżdżonego słownika

Dane:

```
raport = {
    'Q1': {'przychod': 120_000, 'koszt': 80_000},
    'Q2': {'przychod': 145_000, 'koszt': 92_000},
    'Q3': {'przychod': 138_000, 'koszt': 88_000},
    'Q4': {'przychod': 167_000, 'koszt': 101_000}
}
```

1. Wypisz w osobnych liniach przychód każdego kwartału (bez pętli – odwołuj się jawnie).
2. Oblicz zysk każdego kwartału (`przychod - koszt`) i zapisz go jako nowy klucz `zysk` w słowniku wewnętrznym (`raport['Q1']['zysk'] = ...`, itd. – 4 linijki).
3. Wypisz tabelę w formacie: `Q1 | 120000 | 80000 | 40000` (użyj `:^6` lub `:>8` do wyrównania).
4. Zapisz listę wszystkich zysków i wypisz ich sumę, minimum, maksimum oraz średnią.

Zadanie 12. Pełne studium pojedynczego rekordu

Dany jest rekord datasetu Iris w postaci krotki:

```
rekord = ('setosa', 5.1, 3.5, 1.4, 0.2)
```

Pierwszy element to nazwa gatunku, kolejne cztery – kolejno `sepal_length`, `sepal_width`, `petal_length`, `petal_width`.

1. Rozpakuj rekord do zmiennych: `gatunek, sl, sw, pl, pw`.
2. Utwórz słownik `rekord_slownik`, w którym kluczami są `'species'`, `'sepal_length'`, `'sepal_width'`, `'petal_length'`, `'petal_width'`, a wartościami odpowiednie pola krotki.
3. Wypisz w formacie tabelarycznym:

```
species      : setosa
   sepal_length :   5.1
   sepal_width  :   3.5
   petal_length :   1.4
   petal_width  :   0.2
```

(nazwa klucza wyrównana do lewej na 13 znakach, liczba do prawej na 5 z jednym miejscem po przecinku).

1. Używając `sum` i slajsu krotki, wypisz sumę wszystkich czterech cech liczbowych (`sum(rekord[1:])`).

🔴 Zadania zaawansowane

Zadanie 13. Porównanie trzech modeli w formie tabeli

Dane:

```
wyniki = {
    'LogReg':       {'acc': 0.9667, 'prec': 0.9673, 'rec': 0.9667, 'f1': 0.9667},
    'DecisionTree': {'acc': 0.9333, 'prec': 0.9342, 'rec': 0.9333, 'f1': 0.9332},
    'RandomForest': {'acc': 0.9600, 'prec': 0.9610, 'rec': 0.9600, 'f1': 0.9601}
}
```

Bez użycia pętli zbuduj (i wypisz) raport:

```
============================================================
Model          Acc     Prec     Rec      F1
```

```
============================================================
LogReg          0.9667  0.9673  0.9667  0.9667
DecisionTree    0.9333  0.9342  0.9333  0.9332
RandomForest    0.9600  0.9610  0.9600  0.9601
============================================================
Średnia acc: 0.9533
Średnia f1 : 0.9533
```

Wskazówki:

- Pobieraj wartości jawnie: `wyniki['LogReg']['acc']` itd.
- Użyj `sorted` i mnożenia stringa (`'=' * 60`).
- Średnie: `(wyniki['LogReg']['acc'] + wyniki['DecisionTree']['acc'] + wyniki['RandomForest']['acc']) / 3` .

Zadanie 14. Zbiór cech vs lista wartości

Dana jest lista rekordów datasetu (każdy rekord to krotka `(gatunek, sl, sw, pl, pw)`):

```
dane = [
    ('setosa', 5.1, 3.5, 1.4, 0.2),
    ('setosa', 4.9, 3.0, 1.4, 0.2),
    ('versicolor', 7.0, 3.2, 4.7, 1.4),
    ('versicolor', 6.4, 3.2, 4.5, 1.5),
    ('virginica', 6.3, 3.3, 6.0, 2.5),
    ('virginica', 5.8, 2.7, 5.1, 1.9),
    ('setosa', 4.7, 3.2, 1.3, 0.2)
]
```

1. Używając slajsu (`[0]`) oraz konstruktora `set()` na **liście rozumień** NIE WOLNO (bo to lekcja 8). Zamiast tego odwołaj się do elementów jawnie:

```
gatunki = {dane[0][0], dane[1][0], dane[2][0], dane[3][0], dane[4][0], dane[5][0], dane[6][0]}
```

Wypisz zbiór `gatunki` .

1. Wypisz liczbę unikalnych gatunków (`len`).
2. Utwórz krotkę `pierwszy_rekord = dane[0]` i rozpakuj ją.
3. Utwórz listę `sepal_length` zawierającą drugie pole (`[1]`) z każdego rekordu – znowu jawnie, bez pętli:

```
sepal_length = [dane[0][1], dane[1][1], dane[2][1], dane[3][1],
                dane[4][1], dane[5][1], dane[6][1]]
```

Wypisz: minimum, maksimum, sumę i średnią (do 3 miejsc po przecinku).

Zadanie 15. Trzy modele jako tabela w dwuwymiarowym słowniku

Kontynuujemy dane z zadania 13. Zbuduj „tabelę transponowaną" – słownik, którego kluczami są metryki, a wartościami słowniki model→wartość:

```
{
  'acc':  {'LogReg': 0.9667, 'DecisionTree': 0.9333, 'RandomForest': 0.9600},
  'prec': {...},
  'rec':  {...},
  'f1':   {...}
}
```

1. Zbuduj ten słownik jawnie (4 klucze, każdy z trzema wpisami – użyj istniejącego `wyniki`).
2. Wypisz dla każdej metryki: nazwę metryki, **najlepszy model** (użyj `max(slownik, key=slownik.get)`) oraz jego wartość. Tak – `max` ze słownikiem i argumentem `key=d.get` działa bez pętli i bez `lambda`:

```
najlepszy_acc = max(tabela['acc'], key=tabela['acc'].get)
```

(Przykład wyjścia: `acc: LogReg (0.9667)`.) Powtórz dla czterech metryk.

1. Zbuduj **zbiór nazw modeli**, które choć raz były najlepsze w jakiejś metryce. Jeśli wszystkie cztery najlepsze są tym samym modelem, zbiór będzie jednoelementowy.

Zadanie 16. Analiza sprzedaży z kluczem-krotką

Dane:

```
sprzedaz = {
    (2024, 'Q1'): 120_000,
    (2024, 'Q2'): 145_000,
    (2024, 'Q3'): 138_000,
    (2024, 'Q4'): 167_000,
    (2025, 'Q1'): 130_000,
    (2025, 'Q2'): 155_000,
```

```
    (2025, 'Q3'): 149_000
}
```

1. Wypisz całkowitą sprzedaż (`sum`).
2. Wypisz najwyższą i najniższą sprzedaż (`max` , `min`).
3. Użyj `max(sprzedaz, key=sprzedaz.get)` , aby znaleźć **klucz** (krotkę) z największą sprzedażą. Rozpakuj go: `rok_max, kw_max = ...` . Wypisz: `Najwyższa sprzedaż: 2024 Q4 – 167000` .
4. Użyj `sorted(sprzedaz, key=sprzedaz.get, reverse=True)` – zwróci klucze posortowane wg wartości malejąco. Zapisz wynik do zmiennej i wypisz trzy najwyższe klucze (indeksy 0, 1, 2).
5. Zbuduj zbiór unikalnych **lat** ze wszystkich kluczy. Skorzystaj z jawnego zapisu – klucze masz wypisane w literał słownika.

> 💡 **Wskazówka**
>
> `max(slownik, key=slownik.get)` to bardzo elegancki wzorzec. Wbrew pozorom **nie używa** `lambda` – po prostu przekazuje metodę `.get` jako funkcję kluczującą. To wzorzec dozwolony w tej lekcji.

Zadanie 17. Diagnoza różnic między dwiema wersjami konfiguracji

Dwie wersje konfiguracji modelu:

```
wersja_1 = {'lr': 0.01, 'epochs': 100, 'optimizer': 'sgd',   'batch_size': 32, 'dropout':
0.2}
wersja_2 = {'lr': 0.001, 'epochs': 100, 'optimizer': 'adam', 'batch_size': 64, 'momentum':
0.9}
```

Bez pętli i bez `if` wypisz raport:

1. Zbiór kluczy obu wersji (`set(wersja_1) | set(wersja_2)`).
2. Klucze wspólne (iloczyn zbiorów).
3. Klucze tylko w wersji 1 (różnica).
4. Klucze tylko w wersji 2 (różnica).
5. Klucze wspólne, w których **wartości się różnią** – zrób to „ręcznie", sprawdzając każdy wspólny klucz jawnie:

```
rozne_lr          = wersja_1['lr'] != wersja_2['lr']
   rozne_epochs      = wersja_1['epochs'] != wersja_2['epochs']
   rozne_optimizer   = wersja_1['optimizer'] != wersja_2['optimizer']
   rozne_batch_size  = wersja_1['batch_size'] != wersja_2['batch_size']
```

Wypisz wartości tych flag (True/False).

Zadanie 18. Mały „JSON" – zagnieżdżona struktura

Zbuduj słownik `projekt` reprezentujący konfigurację eksperymentu:

```
projekt = {
    'nazwa': 'iris_classification',
    'wersja': (1, 2, 0),                        # krotka – wersja jako (major, minor,
patch)
    'autorzy': {'Anna', 'Bartek', 'Cezary'},    # zbiór
    'dataset': {
        'nazwa': 'Iris',
        'shape': (150, 4),
        'klasy': ('setosa', 'versicolor', 'virginica')
    },
    'hiperparametry': {
        'lr': 0.001,
        'batch_size': 64,
        'epochs': 50
    }
}
```

1. Rozpakuj krotkę wersji: `major, minor, patch = projekt['wersja']`. Wypisz w formacie: `v1.2.0`.
2. Rozpakuj shape: `n_rekordow, n_cech = projekt['dataset']['shape']`. Wypisz: `Dataset Iris: 150 × 4`.
3. Dodaj nowego autora `'Dorota'` do zbioru autorów (`.add()`).
4. Wypisz liczbę autorów i posortowaną alfabetycznie listę (`sorted`).
5. Zaktualizuj `hiperparametry` słownikiem `{'dropout': 0.3, 'batch_size': 128}` (wartość `batch_size` ma się nadpisać). Wypisz zaktualizowany słownik hiperparametrów.
6. Zmień **klasy** w dataset z krotki na zbiór (`projekt['dataset']['klasy'] = set(projekt['dataset']['klasy'])`) i sprawdź operatorem `in`, czy `'setosa'` jest w zbiorze.

---

8. 📚 Podsumowanie

W tej lekcji dodaliśmy trzy nowe kolekcje do naszego zestawu narzędzi:

- **Słowniki (`dict`)** – mapowanie klucz→wartość. Kluczowe metody: `.get()` , `.keys()` , `.values()` , `.items()` , `.update()` , `.pop()` , `.setdefault()` . Łączenie operatorem `|` . Świetne do reprezentacji rekordów i metadanych.
- **Zbiory (`set`)** – kolekcja unikalnych elementów. Cztery operacje: suma `|` , iloczyn `&` , różnica `-` , różnica symetryczna `^` . Błyskawiczny test przynależności operatorem `in` . Idealne do deduplikacji i porównywania grup.
- **Krotki (`tuple`)** – niemutowalna sekwencja. Pamiętaj o przecinku w krotce jednoelementowej `(42,)` . Wspaniale sprawdzają się jako klucze słownika i jako rekordy dzięki rozpakowywaniu `a, b = t` .
- **Dobór struktury** – listy do zmiennych sekwencji, krotki do stałych rekordów, słowniki do mapowań, zbiory do unikalności i `in` .
- **Wzorzec bez pętli** – `max(d, key=d.get)` , `sorted(d, key=d.get)` , `sum(d.values())` , `set(a) | set(b)` to w pełni profesjonalne i „pythonic" wzorce, które pozwalają rozwiązywać złożone zadania **bez** pętli i warunków.
- **Green IT** – krotki oszczędzają pamięć, zbiory przyspieszają sprawdzanie `in` , a operatorowe wersje operacji mnogościowych są zarówno czytelne, jak i wydajne.

W **Lekcji 7** poznamy instrukcje warunkowe (`if` , `elif` , `else`) oraz pętle (`for` , `while`). Dopiero wtedy w pełni rozwiniemy to, co już potrafimy z tymi kolekcjami – np. zamiast odwoływać się jawnie do `dane[0][0]` , `dane[1][0]` , `dane[2][0]` , wystarczy jedna pętla.

---

9. 📖 Dodatkowe zasoby

- Python docs: Mapping Types – dict – kompletna dokumentacja słowników.
- Python docs: Set Types – set, frozenset – zbiory i ich niemutowalna wersja `frozenset` .
- Python docs: Tuples and Sequences – krotki w oficjalnym tutorialu Pythona.
- Real Python: Dictionaries in Python – bardzo dobry, praktyczny przewodnik.
- Real Python: Sets in Python – zbiory z przykładami i porównaniem wydajności.
- PEP 448 – Additional Unpacking Generalizations – teoria rozpakowywania i operatorów `*` , `**` .

ROZPOCZNIJ KARIERĘ W DATA SCIENTIST

Gotowy na następny krok?

Ten materiał to tylko początek. Dołącz do kursu Data Scientist Online + AI i zdobądź praktyczne umiejętności pod okiem ekspertów.

- 5,5 miesiąca intensywnej nauki
- Zajęcia na żywo 2x w tygodniu
- Certyfikat + pomoc w znalezieniu pracy

Zobacz szczegóły kursu →

Zadzwoń do nas

info@learnit.com.pl +48 573-580-990 learnit.com.pl